AI 2002: Artificial Intelligence

Intelligent Agents

Spring 2026

2.1 Agents and Environment

- An agent is anything that can be viewed as perceiving its **environment** through **sensors** and Sensor acting upon that environment through **actuators**.

- Agents include humans, robots, softbots (software agents) etc.

- Percept:
  - The content an agent's sensors are perceiving.
- Percept sequence:
  - The complete history of everything the agent has ever perceived.

2.1 Agents and Environment

- Agent function **(f)**
  - **Agent's behavior** that maps any given percept sequence to an action
  - an abstract mathematical description.

$$f: P^* \rightarrow A$$

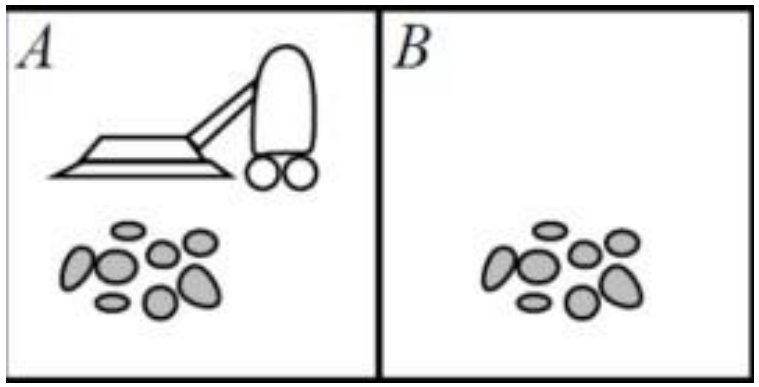
- Agent Program
  - Agent function for an artificial agent will be implemented by an agent program
  - a concrete implementation, running within some physical system to produce f.

- For example:
  - Agent function = Law book (defines ideal behavior)
  - Agent program = Judge (applies rules logically in real time)
  - Table-driven = Cheat-sheet judge (memorized all cases explicitly)

2.1 Agents and Environment

Vacuum- cleaner world example

- Percepts
  - location and contents, e.g. [A; Dirty]
- Actions
  - Left, Right, Suck, NoOp



Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮

Partial tabulation of a simple agent function for the vacuum-cleaner world

2.2 Good Behavior: The Concept Of Rationality

- A rational agent is one that does the right thing
  - -> What is the right thing?
  - A sequence of actions causes the environment to go through a sequence of states.
  - If the sequence is desirable, then the agent has performed well.

- Desirability is captured by **performance measure** that evaluates any given sequence of environment states.
  - A rational agent chooses whichever action **maximizes the expected value of the performance measure** given the percept sequence.
  - Consider the vacuum cleaner agent, we might propose to measure performance by the amount of dirt cleaned up in a single eight-hour shift. With a rational agent, what you ask for is what you get.

2.2 Good Behavior: The Concept Of Rationality

- **Rationality** is NOT the same as perfection.
  - An omniscient (perfect) agent knows the actual outcome of its actions and can act accordingly. But perfection is impossible in reality.
  - Rationality maximizes expected performance, while perfection maximizes actual performance.

- A rational agent not only to gather information (exploration) but also to **learn** as much as possible from what it perceives.
  - The incorporation of learning allows one to design a single rational agent that will succeed in a vast variety of environments.
- An agent relies on the prior knowledge of its designer rather than on its own percepts, we say that the agent lacks **autonomy**.
  - A rational agent should be autonomous.
  - It should learn what it can to compensate for partial or incorrect prior knowledge.
  - For example, a vacuum-cleaning agent that learns to predict where and when additional dirt will appear will do better than one that does not.

2.3 The Nature of Environments

- To design a rational agent, we must specify the **task environment**.
- The task environment is mostly referred as PEAS (Performance measure, Environment, Actuators, and Sensors)

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits, minimize impact on other road users	Roads, other traffic, police, pedestrians, customers, weather	Steering, accelerator, brake, signal, horn, display, speech	Cameras, radar, speedometer, GPS, engine sensors, accelerometer, microphones, touchscreen

Figure 2.4 PEAS description of the task environment for an automated taxi driver.

2.3 The Nature of Environments

The **environment type** largely determines the agent design.

- Fully observable vs Partially observable:
  - If an agent's sensor give it access to the complete state of the environment at each point in time, then task environment is called as fully observable
  - Fully observable environments are convenient because the agent need not maintain any internal state to keep track of the world.
  - An environment might be partially observable because of noisy and inaccurate sensors.
- Single Agent vs Multi Agent:
  - Solving a crossword puzzle is a single agent environment, whereas an agent playing chess is in a two agent environment.
- Competitive vs Cooperative
  - Chess is competitive multi agent environment because an agent tries to maximize its performance while minimizing the performance of other agent.
  - Taxi-driving is partially cooperative multi agent environment because avoiding collisions maximizes all agent's performances.

2.3 The Nature of Environments

- Deterministic vs Stochastic:
  - if the next state of the environment is completely determined by the current state and the action executed by the agent, then the environment is deterministic otherwise, it is stochastic.
- Episodic vs Sequential:
  - In an episodic task environment, the agent's experience is divided into atomic episodes. In each episode the agent receives a percept and then performs a single action. The next episode does not depend on the actions taken in previous episodes.
  - In sequential environment, on the other hand, the current decision could affect all future decisions. Chess and taxi driving are sequential: in both cases, short-term actions can have long-term consequences.
- Static vs Dynamic:
  - if the environment can change while an agent is deliberating, then we say the environment is dynamic for that agent; otherwise, it is static.
- Discrete vs Continuous:
  - The discrete/continuous distinction applies to the state of the environment.
  - Chess is discrete; Taxi-driving continuous.

2.3 The Nature of Environments

- The real world is (of course) partially observable, stochastic, sequential, dynamic, continuous, multi-agent

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

2.4 The Structure of Agents

- The four basic agent types in order of increasing generality:
  - simple reflex agents
  - reflex agents with state
  - goal-based agents
  - utility-based agents
- All these can be turned into learning agents.

Table-driven agent

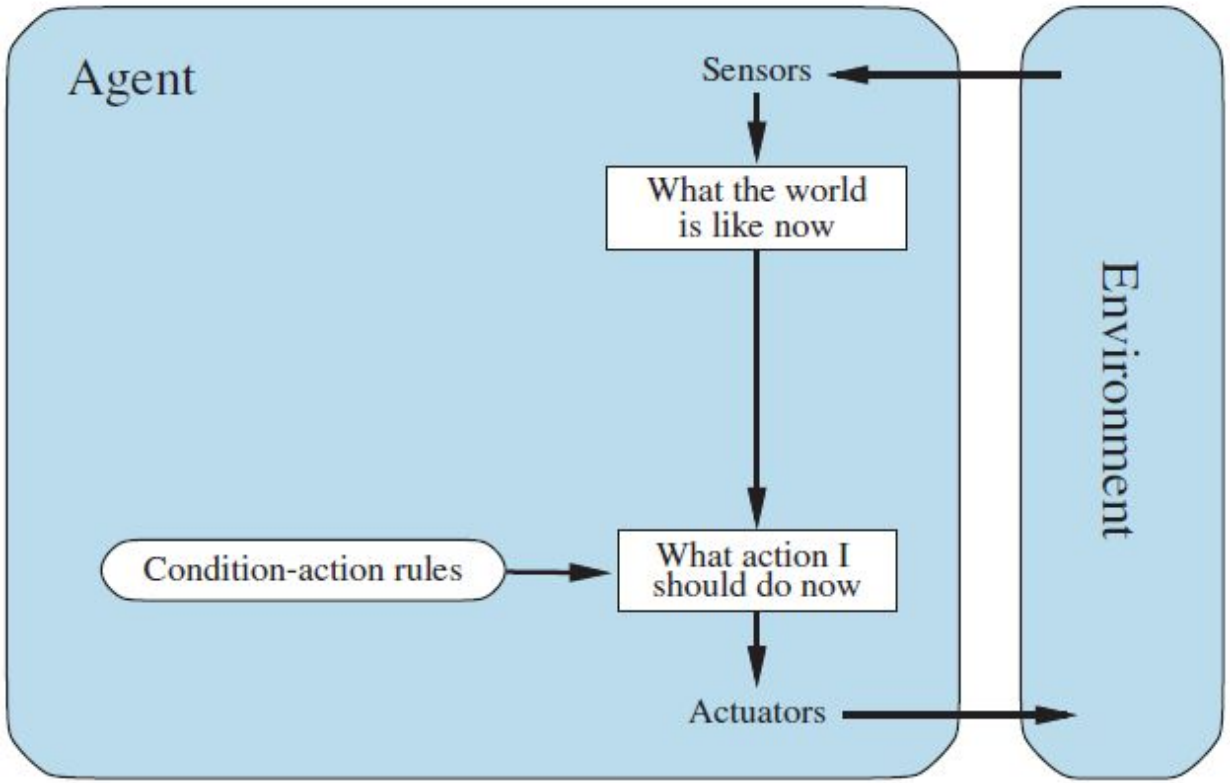
```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
              table, a table of actions, indexed by percept sequences, initially fully specified

  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

Figure 2.7 The TABLE-DRIVEN-AGENT program is invoked for each new percept and returns an action each time. It retains the complete percept sequence in memory.

Percept sequence	Action
[*A*, *Clean*]	*Right*
[*A*, *Dirty*]	*Suck*
[*B*, *Clean*]	*Left*
[*B*, *Dirty*]	*Suck*
[*A*, *Clean*], [*A*, *Clean*]	*Right*
[*A*, *Clean*], [*A*, *Dirty*]	*Suck*
⋮	⋮
[*A*, *Clean*], [*A*, *Clean*], [*A*, *Clean*]	*Right*
[*A*, *Clean*], [*A*, *Clean*], [*A*, *Dirty*]	*Suck*
⋮	⋮

2.4.2 Simple Reflex Agents

- Select actions on the basis of the current percept, ignoring the rest of the percept history.



Agent Program

function SIMPLE-REFLEX-AGENT(*percept*) **returns** an action
  persistent: *rules*, a set of condition–action rules

  state ← INTERPRET-INPUT(*percept*)
  rule ← RULE-MATCH(*state*, *rules*)
  action ← *rule*.ACTION
  return *action*

Example Agent Function

"if dirty → Suck, else → Move"

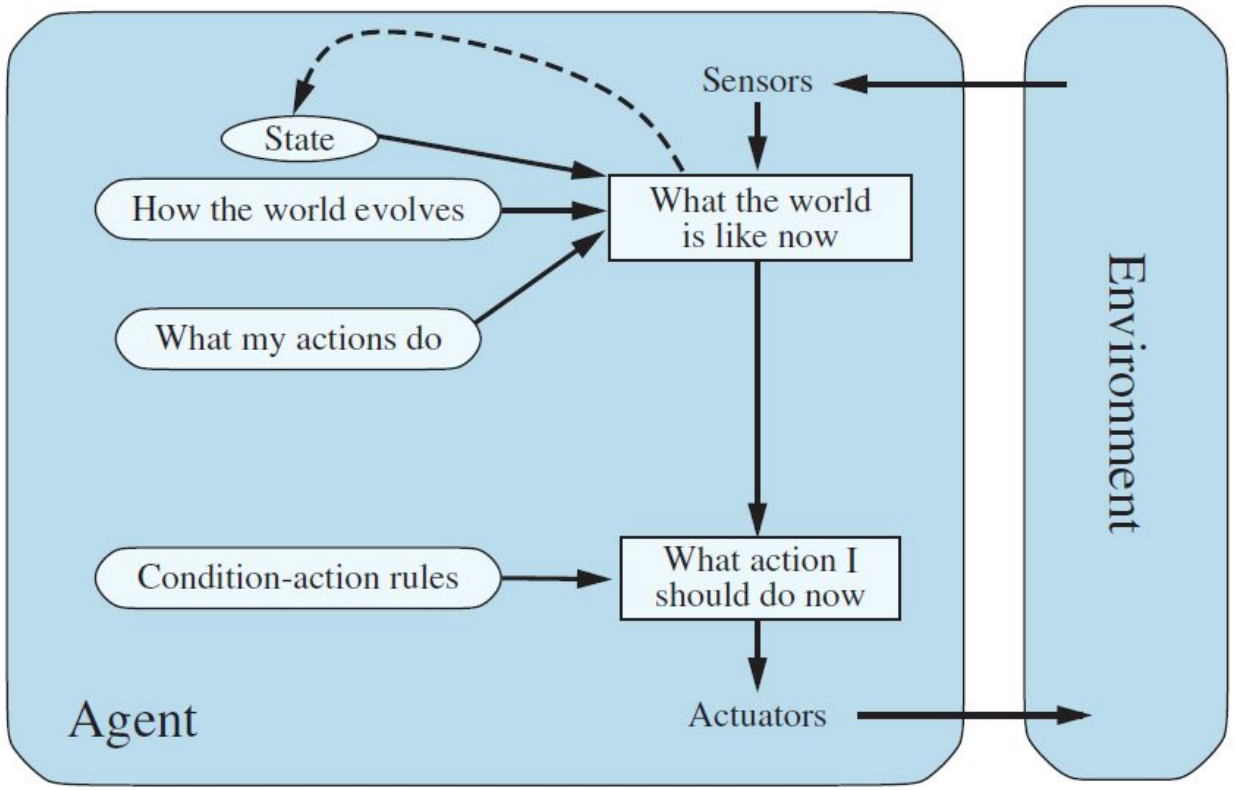
Example Agent Program

function REFLEX-VACUUM-AGENT([*location*,*status*]) **returns** an action

  if *status* = *Dirty* **then return** *Suck*
  else if *location* = *A* **then return** *Right*
  else if *location* = *B* **then return** *Left*

2.4.3 Model-based reflex agents

- Keeps track of the current state of the world, using an internal model. It then chooses an action in the same way as the reflex agent

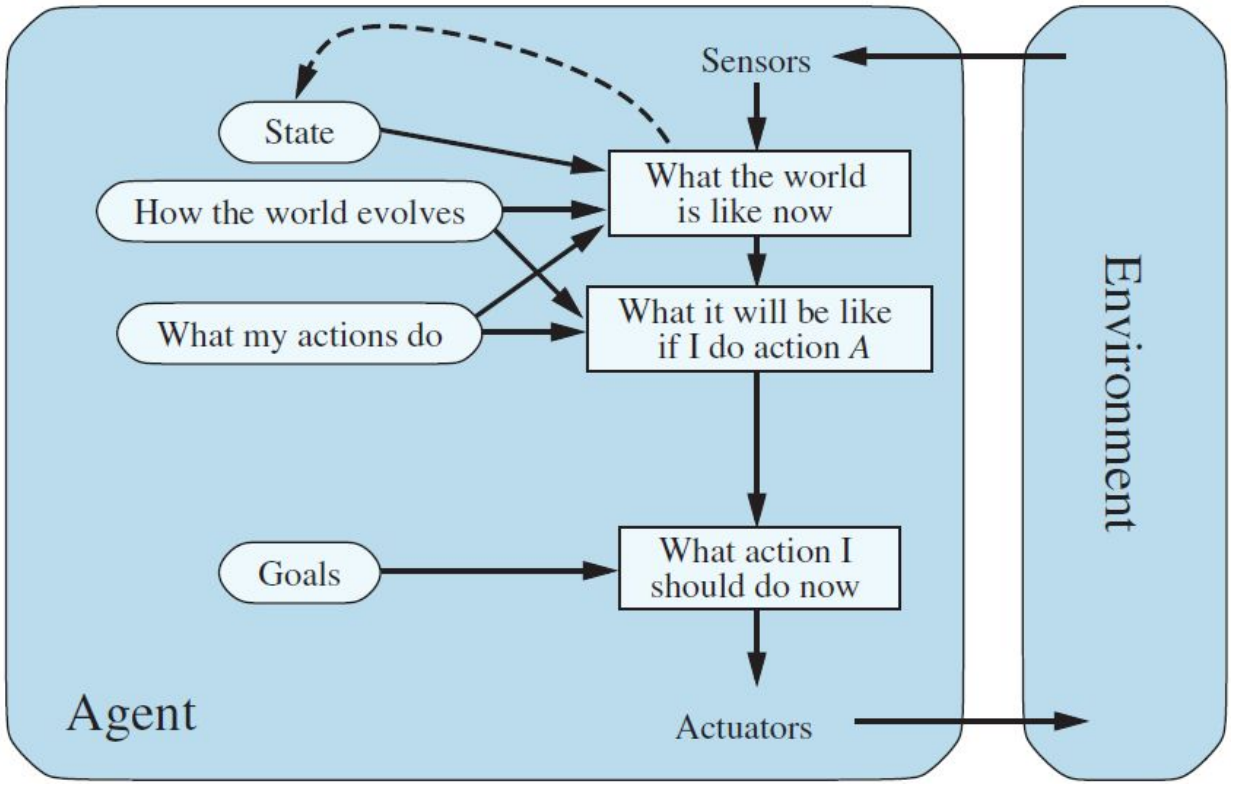


function MODEL-BASED-REFLEX-AGENT(*percept*) **returns** an action
  persistent: *state*, the agent's current conception of the world state
    transition_model, a description of how the next state depends on the current state and action
    sensor_model, a description of how the current world state is reflected in the agent's percepts
    rules, a set of condition–action rules
    action, the most recent action, initially none

  state ← UPDATE-STATE(*state*, *action*, *percept*, *transition_model*, *sensor_model*)
  rule ← RULE-MATCH(*state*, *rules*)
  action ← *rule*.ACTION
  return *action*

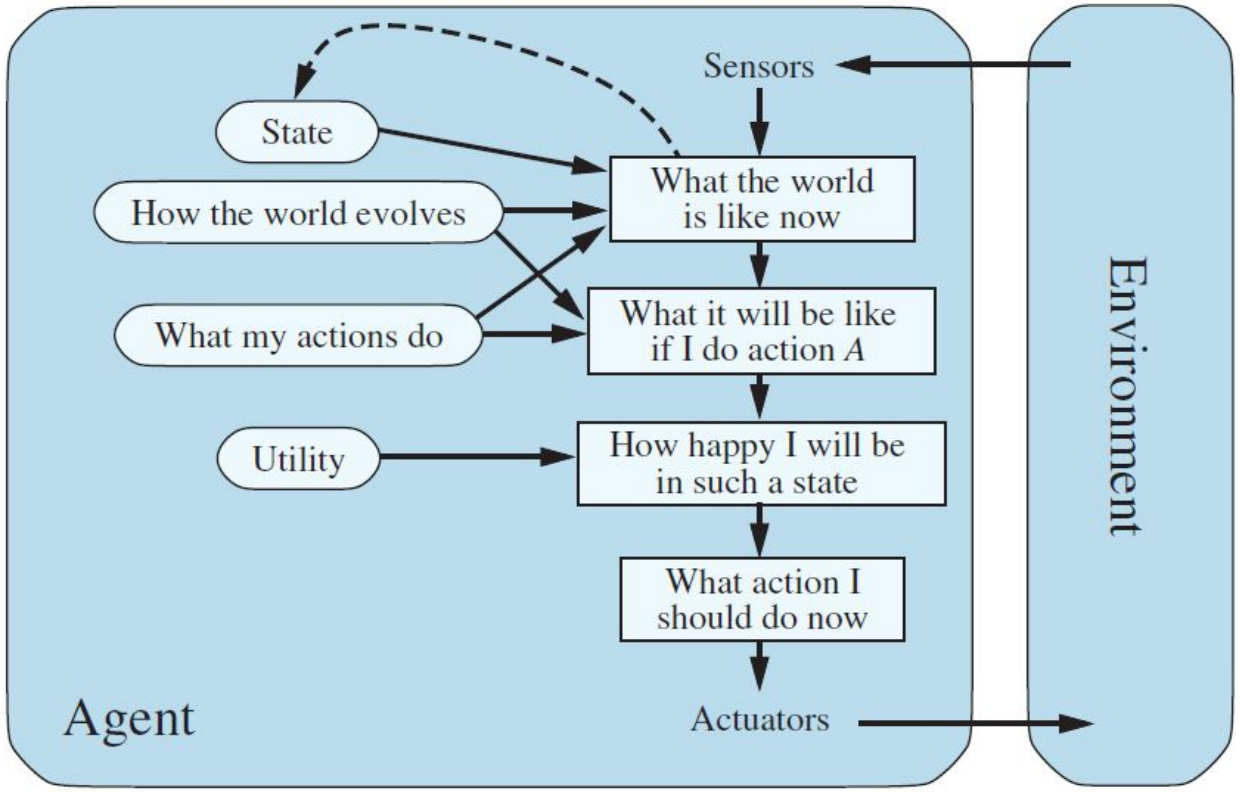
2.4.4 Goal-based agents

- Keeps track of the world state as well as a set of goals it is trying to achieve, and chooses an action that will (eventually) lead to the achievement of its goals.



2.4.5 Utility-based agents

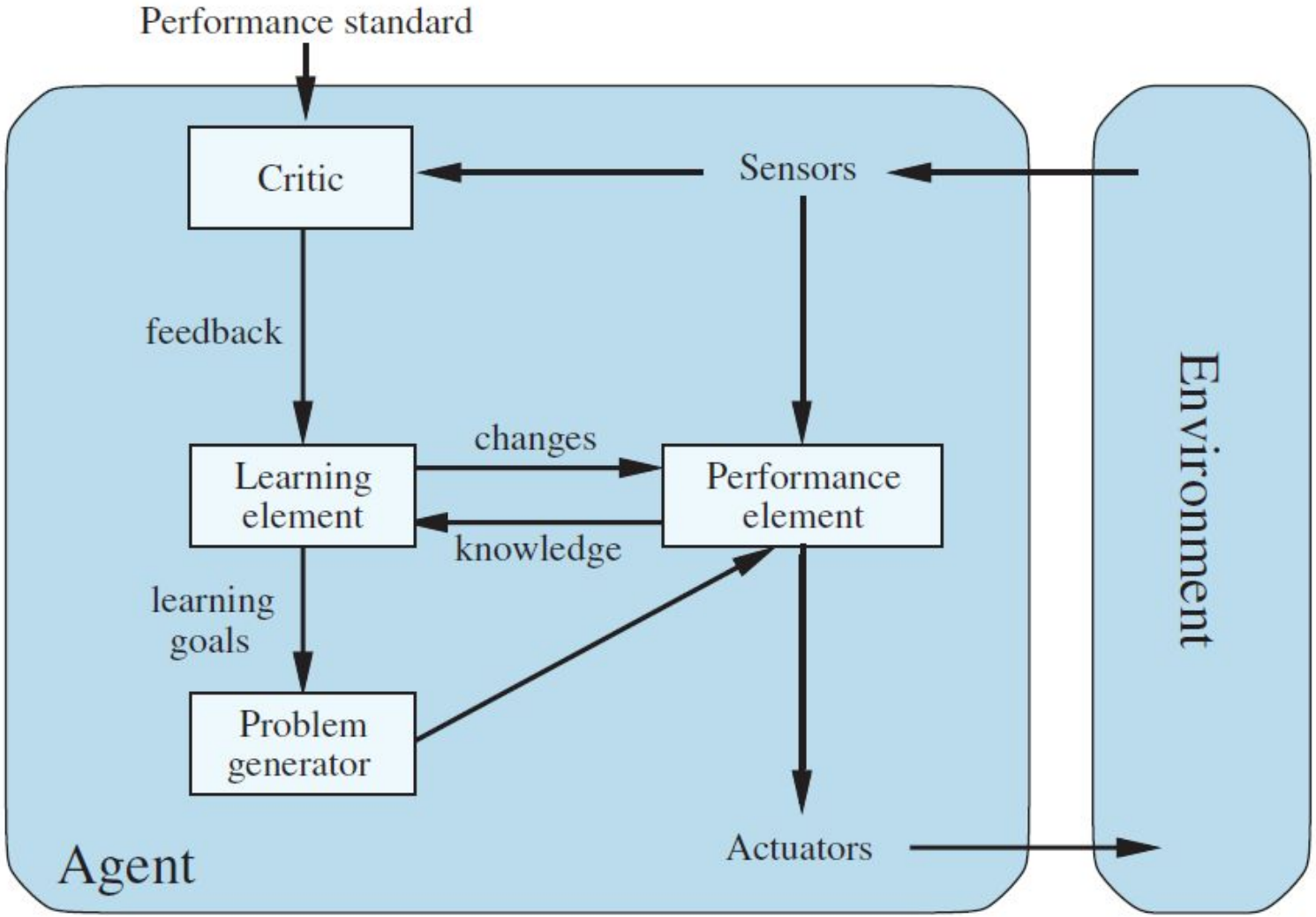
- Uses a model of the world, along with a utility function that measures its preferences among states of the world.



- Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.

2.4.6 Learning agents

- **Learning element** is responsible for making improvements.
- **Performance element** is responsible for selecting external actions.
- The learning element uses feedback from **critic** on how agent is doing and determines how performance element should be modified to do better in future.
- **Problem generator** is responsible for suggesting actions that will lead to new and informative experiences.



Problem Representation In AI

- One of the core aspects of AI is problem representation, which plays a crucial role in how machines understand and solve complex problems. In simple terms, problem representation refers to the process of transforming real-world situations or scenarios into a format that can be comprehended and processed by machines.

- Representation in AI is a fundamental concept that enables machines to reason, learn, and make decisions. It involves encoding various aspects and attributes of a problem, such as objects, relationships, constraints, and goals, into a structured format that can be manipulated and analysed by AI algorithms.